JAVA NOTES — TOKENS & IDENTIFIERS

10-page handwritten-style revision notes • Core Java language fundamentals

1. What is a token?

A token is the smallest meaningful unit of a Java program that the compiler recognises. Just as a sentence is made from words and punctuation, Java source code is made from tokens.

```
int total = price + 5;
   |    |     |   |   |
 keyword identifier operator identifier literal
```

Main categories of Java tokens

- *Keywords: reserved words with a predefined meaning, such as* `class`, `int` *and* `if`.
- *Identifiers: programmer-chosen names for classes, methods, variables, packages, etc.*
- *Literals: fixed values written directly in code, such as* `42`, `"Hello"` *and* `true`.
- *Operators: symbols that perform operations, such as* `+`, `==` *and* `&&`.
- *Separators / punctuators: symbols that organise code, such as* `()`, `{}`, `[]`, `;` *and* `,`.

Token example

```
public class Demo {
    public static void main(String[] args) {
        int age = 20;
        System.out.println(age);
    }
}
```

In this example, `public`, `class`, `static` *and* `int` *are keywords;* `Demo`, `main`, `args` *and* `age` *are identifiers;* `20` *is a literal.*

Remember: Whitespace and comments usually separate or explain tokens; they are not themselves Java tokens in the same sense as identifiers or operators.

2. IDENTIFIERS

Names created by the programmer

Meaning

An identifier is the name used to identify a program element: a variable, method, class, interface, enum, record, package or label.

Rules for writing identifiers

- *The first character must be a Java letter, a currency symbol such as* `$`*, or a connecting punctuation character such as* `_`*.*
- *After the first character, digits are allowed too.*
- *A space is not allowed inside an identifier.*
- *A keyword cannot be used as an ordinary identifier.*
- *Identifiers are case-sensitive:* `total`*,* `Total` *and* `TOTAL` *are different names.*
- *A single underscore* `_` *is reserved and cannot be used as an identifier in modern Java.*

Valid and invalid examples

```
// Valid
studentName
_marks
$amount
number2
café

// Invalid
2number       // begins with a digit
student name  // contains a space
class         // keyword
first-name    // hyphen is not allowed
_             // single underscore is reserved
```

Naming conventions (recommended)

- *Class / interface: use PascalCase —* `StudentRecord`*,* `Runnable`*.*
- *Variable / method: use lower camelCase —* `totalMarks`*,* `calculateTax()`*.*
- *Constants: use uppercase words separated by underscores —* `MAX_SIZE`*.*
- *Packages: usually lowercase —* `com.example.app`*.*

Remember: A naming convention is a style recommendation, not usually a compiler rule. Clear names make code easier to read and maintain.

3. KEYWORDS & RESERVED WORDS

Words whose meaning is defined by the Java language

Keywords

A keyword has a special meaning in Java syntax. It cannot be used as the name of a variable, class or method.

Common keyword groups

- *Access / class design:* `public`, `private`, `protected`, `class`, `interface`, `extends`, `implements`.
- *Data types:* `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`, `void`.
- *Control flow:* `if`, `else`, `switch`, `case`, `for`, `while`, `do`, `break`, `continue`, `return`.
- *Object / exception:* `new`, `this`, `super`, `try`, `catch`, `finally`, `throw`, `throws`.
- *Modifiers / other:* `static`, `final`, `abstract`, `synchronized`, `volatile`, `transient`, `native`, `strictfp`, `assert`, `enum`, `package`, `import`.

Reserved but not used as ordinary keywords

Java also has reserved words that are not used as normal keywords in current Java syntax. `const` *and* `goto` *are reserved but have no current language function. The words* `true`, `false` *and* `null` *are literals, not keywords.*

Contextual keywords

Some words act as keywords only in particular contexts. Examples include `var`, `record`, `sealed`, `permits` *and* `yield`. *Their meaning depends on where they appear and on the Java language version.*

> *Remember: Do not memorise only a short list: check the Java version used by your compiler, because the language adds contextual keywords over time.*

4. LITERALS

Values written directly in source code

Meaning and types

A literal is a fixed value represented directly in Java code. It is assigned or used without first calculating it from another expression.

Kinds of literals

- *Integer:* `25`, `-8`, `0`.
- *Floating-point:* `3.14`, `2.5f`, `6.02e23`.
- *Character:* `'A'`, `'7'`, `'\n'`.
- *String:* `"Java"`, `"Hello World"`.
- *Boolean:* `true` *and* `false`.
- *Null:* `null`, *meaning no object reference.*

Escape sequences

Escape sequences begin with a backslash and represent special characters inside character or string literals.

```
'\n'   new line       '\t'   horizontal tab
'\r'   carriage return '\b' backspace
'\\'   backslash      '\'' single quote
'\"'   double quote   '\u0041' Unicode escape (A)
```

Text blocks

For multi-line text, Java supports text blocks, opened and closed with three double quotes. They are useful for readable multi-line strings.

```
String message = """
    Welcome to Java
    Learn one concept at a time.
    """;
```

Remember: A character literal uses single quotes and contains one UTF-16 code unit; a String literal uses double quotes and can contain text.

5. NUMERIC LITERALS

Integer and floating-point values

Integer literal forms

Java lets you write integer values in four common number systems:

```
int decimal = 26;       // base 10
int binary  = 0b11010;   // base 2
int octal   = 032;       // base 8
int hex     = 0x1A;      // base 16
```

All four examples represent the decimal value 26. Binary uses the prefix `0b`*, octal begins with* `0`*, and hexadecimal uses* `0x`*.*

Integer types and suffix

- *An unsuffixed decimal integer literal is generally of type* `int` *if it fits.*
- *Use* `L` *or* `l` *for a* `long` *literal; uppercase* `L` *is easier to read.*

```
long population = 8_000_000_000L;
int score = 1_000_000;
```

Floating-point literals

- *A decimal point or exponent can make a floating-point literal.*
- *By default, a floating-point literal is* `double`*.*
- *Append* `f` *or* `F` *to make it a* `float`*.*

```
double pi = 3.1415926535;
float rate = 7.5f;
double distance = 1.2e3; // 1200.0
```

Underscores and common mistakes

Underscores may improve readability between digits: `1_000_000`*. They cannot be placed at the beginning or end of a number, beside a decimal point, or immediately before a suffix.*

```
int amount = 12_500;  // valid
// int x = _1000;      // invalid numeric form
// float f = 2.5;      // error: double cannot narrow implicitly
float f = 2.5f;        // correct
```

Remember: Floating-point values are approximations for many decimal fractions. For exact decimal financial calculations, consider BigDecimal.

6. PRIMITIVE DATA TYPES

Eight built-in value types

The eight primitive types

Primitive variables store simple values directly. Java has eight primitive types:

Type	Size	Range / values	Typical use
byte	8-bit	-128 to 127	small integers
short	16-bit	-32,768 to 32,767	small-range integers
int	32-bit	-2^{31} to $2^{31}-1$	default whole-number choice
long	64-bit	-2^{63} to $2^{63}-1$	large whole numbers
float	32-bit	approx. 6–7 decimal digits	single precision
double	64-bit	approx. 15–16 decimal digits	double precision
char	16-bit	0 to 65,535 code units	one UTF-16 code unit
boolean	JVM-dependent storage	true or false	logical value

Example declarations

```
byte level = 10;
int marks = 95;
long views = 9_000_000_000L;
float temperature = 36.5f;
double average = 87.625;
char grade = 'A';
boolean passed = true;
```

Important points

- Use `int` for most everyday integer calculations and `double` for most decimal calculations.
- A `char` is not the same as a one-character String.
- `boolean` accepts only `true` or `false`; it is not an integer.

Remember: The exact memory representation of boolean is JVM-dependent; Java specifies its values, not a fixed bit size.

7. REFERENCE TYPES

Variables that refer to objects

Primitive vs reference

A reference-type variable holds a reference to an object (or `null`), rather than containing the object's full data as a primitive value.

Examples of reference types

- *Classes:* `String`, `Scanner`, `Student`.
- *Arrays:* `int[]`, `String[]`.
- *Interfaces:* `Runnable`, `List`.
- *Enums, records and other class-based types.*

Creating and using an object

```
String language = "Java"; // String reference
int[] marks = {80, 90, 95}; // array reference

Student s = new Student(); // object created
s.name = "Asha";
```

The variable `s` refers to a Student object. More than one reference can point to the same object.

```
Student first = new Student();
Student second = first;
second.name = "Ravi";
// first.name is also "Ravi"
```

null reference

The literal `null` means that a reference variable currently refers to no object.

```
String name = null;
if (name == null) {
    System.out.println("No name assigned");
}
```

Trying to call a method through a null reference causes a NullPointerException.

Quick comparison

- *Primitive: value such as `int n = 10;`.*
- *Reference: reference to an object such as `String s = "Hi";`.*
- *Both primitive and reference variables can be local, instance or static variables.*

Remember: Java is always pass-by-value. When an object reference is passed to a method, the copied value is the reference.

8. LOCAL, INSTANCE & STATIC VARIABLES

Variable categories based on where they are declared

Local variables

- *Declared inside a method, constructor or block.*
- *Available only within that declared scope.*
- *Must be definitely assigned before they are read; they do not receive automatic default values.*

Instance variables

- *Declared in a class but outside methods, constructors and blocks, without* `static`.
- *Each object has its own separate instance fields.*
- *Receive default values when an object is created.*

Static (class) variables

- *Declared in a class with the* `static` *keyword.*
- *One class-level variable is shared by instances of that class.*
- *Normally accessed using the class name.*

```
class Student {
    String name;                // instance variable
    static String school = "ABC"; // static variable

    void printDetails() {
        int score = 90;         // local variable
        System.out.println(name + score);
    }
}
```

Comparison

Feature	*Local*	*Instance*	*Static*
Declared	*inside method/block*	*in class, no static*	*in class with static*
Belongs to	*block / method call*	*each object*	*class*
Default value	*none*	*yes*	*yes*
Access	*within scope*	*through object*	*class name preferred*

Remember: A static field is shared; an instance field belongs to each object. A local variable exists only while its scope is active.

9. SCOPE, LIFETIME & final

Where a variable can be used — and how long it exists

Scope

Scope is the region of source code where a name can be accessed.

- *Block scope: a variable declared inside* `{ }` *is available only inside that block.*
- *Method scope: a local variable is usable within its method or the relevant nested blocks.*
- *Parameter scope: a method or constructor parameter is available within that executable body.*
- *Field scope: instance and static fields are accessible according to class and access rules.*

```
if (true) {
    int count = 5;
    System.out.println(count); // OK
}
// System.out.println(count); // error
```

Lifetime

Lifetime means how long a variable or object remains available during program execution. A local variable's scope ends when its block finishes. An instance field exists as part of its object. A static field is associated with the class while that class is loaded.

Do not confuse scope with lifetime: scope is about where code can refer to a name; lifetime is about existence during execution.

The final keyword

A variable declared `final` *can be assigned only once. After it has been assigned, it cannot be reassigned.*

```
final int MAX_USERS = 100;
// MAX_USERS = 200; // compile-time error

final StringBuilder text = new StringBuilder("Hi");
text.append("!");       // allowed
// text = new StringBuilder(); // not allowed
```

For a reference variable, `final` *prevents changing the reference, not modifying the object it refers to.*

Constants

A constant is commonly declared using `static final`*. By convention, its name uses uppercase letters and underscores.*

```
class Settings {
    static final double TAX_RATE = 0.18;
    static final int MAX_RETRIES = 3;
}
```

Remember: A blank final field must be assigned exactly once, commonly in a constructor or initializer. A final local variable must be assigned before use.

10. UNICODE + QUICK REVISION

Java text, character representation and exam practice

Unicode in Java

Unicode is a universal character standard designed to represent text from many writing systems, symbols and emoji. Java source code uses Unicode, so identifiers and character data can include many language characters.

char and UTF-16

Java's `char` *is a 16-bit UTF-16 code unit. Many common characters fit in one* `char`*, but some Unicode characters (including many emoji) need a pair of code units called a surrogate pair.*

```
char letter = 'A';
String word = "□□□□□□";
String emoji = "□";

System.out.println(word);
System.out.println(emoji);
System.out.println(emoji.length()); // often 2
```

For a Unicode code point beyond the Basic Multilingual Plane, use code-point-aware String methods such as `codePointAt()` *and* `codePoints()`*.*

Unicode escapes

A Unicode escape uses `\u` *followed by four hexadecimal digits in source code, for example* `\u0041` *represents the letter A. Unicode escapes are processed early in Java's source translation.*

One-minute revision

- *Token = smallest meaningful unit; identifier = programmer-defined name.*
- *Keywords are reserved by language syntax;* `true`*,* `false` *and* `null` *are literals.*
- *Integer literals can be decimal, binary, octal or hexadecimal; use* `L` *for long.*
- *Eight primitive types: byte, short, int, long, float, double, char, boolean.*
- *Reference variables refer to objects or null.*
- *Local variables need definite assignment; fields get default values.*
- *Instance fields are per object; static fields are shared at class level.*
- *Scope = where a name is usable; lifetime = how long it exists.*
- *final = assign once; static final is a common constant pattern.*

Practice questions

- *1. Is* `null` *a keyword or a literal? Answer: literal.*
- *2. Which suffix makes a floating literal float? Answer:* `f` *or* `F`*.*
- *3. Which variable is shared by all objects? Answer: static field.*
- *4. Does final StringBuilder stop append()? Answer: No; it stops reassignment of the reference.*
- *5. Can one Java char represent every Unicode character? Answer: No; some need two UTF-16 code units.*

Suggested practice: compile the examples, change values, and observe compiler errors. Understanding the rule is more useful than memorising the line.